Chunked Prefill or Disaggregation?

Prefill–Decode Serving over TCP on Single-GPU Cloud Instances

Sunghyun Jun
Independent Researcher
[contact email]

[draft v1, 2026-09-30]

Abstract

Prefill–decode (PD) disaggregation is usually deployed over RDMA or NVLink. The cheapest single-GPU cloud instances have neither, so the KV cache must cross ordinary TCP. We ask when disaggregation over TCP pays off, and against which baseline. On AWS g6e, g7, and g7e 2xlarge instances we served Gemma 4 26B (mixture of experts) and 31B (dense) with stock vLLM, disaggregated with NIXL/UCX over TCP, and judged every request by its time to first token (TTFT) and its longest gap between output tokens (303 audited runs). Colocated replicas stall whenever another request's prefill shares a scheduler step, so their longest gap is set by the chunked-prefill size: for 26B with 4K-token inputs it fell from 48 to 28 ms as the chunk shrank from 2,560 to 1,024 tokens. Against replicas at a 2,560-token chunk, disaggregation served about 3× more requests per GPU under a 50 ms gap limit. Against replicas at the best chunk we measured, it served the same or less: for 26B with 4K inputs, 3–4 requests/s per GPU against the replicas' 4; for 31B chat, 4 against at least 6. Replicas also always delivered the first token sooner. Disaggregation remained the only option when the gap limit was tighter than chunking reached (30 ms for 26B, 30–40 ms for its FP8 variant). Over TCP its first-token cost is mostly the KV transfer. UCX's default 8 KB TCP segment slowed that transfer 3.4×, and NIXL spread each transfer over four TCP connections, which lifted it 1.5–1.8× above the 9.5 Gbps per-flow cap. We release the measurement harness and per-request data.

1 Introduction

Serving a request to a large language model takes two phases: *prefill*, which processes the whole input at once, and *decode*, which produces output tokens one at a time. When both share a GPU, a prefill that is scheduled into the same step as ongoing decodes lengthens that step, and every streaming request on that GPU pauses. Two remedies exist. *Chunked prefill* splits a long prefill into pieces that are batched with decodes, which bounds how much work any one step carries [1]. *Disaggregation* runs the two phases on different GPUs and ships the KV cache from the prefill GPU to the decode GPU [2–4].

Disaggregation pays for its isolation with a KV-cache transfer, and published systems run that transfer over fast links: InfiniBand [3], RDMA [4], or NVLink inside one node [2]. The cheapest single-GPU cloud instances offer none of these. On AWS, for example, the 2xlarge sizes of the g6e, g7, and g7e families have one GPU, no Elastic Fabric Adapter (EFA), and no ENA Express [5]; the 8xlarge sizes of the same families add EFA at roughly 1.6–2× the hourly price [6, 7]. Public documentation discourages disaggregation over TCP: llm-d calls it "extremely slow" and "targeted for local development" [8], and NVIDIA Dynamo warns of a 200–500× slower TTFT without RDMA and recommends aggregated serving instead [9]. Few public measurements show how slow

the TCP path actually is, or whether disaggregation over it still beats a well-configured colocated deployment.

We measured both remedies on the same hardware. We served two Gemma 4 models with stock vLLM on one to three single-GPU instances, disaggregated them with vLLM's NIXL connector over UCX's TCP transport, and judged each request by its TTFT and by the longest gap between its output tokens. We first compared disaggregation against replicas at the chunk size our stack initially accepted. A mock peer review of our first draft pointed out that this baseline was untuned, so we added a session that swept the replicas' chunk size. That sweep changed the paper's conclusion, and we report both baselines. We ask three research questions (RQs):

- **RQ1.** Under a per-request limit on the longest token gap, what sets the stalls of colocated replicas, and how far does chunked prefill go?
- **RQ2.** Over TCP, what does disaggregation cost in TTFT, and what determines that cost?
- **RQ3.** At equal GPU count, when does disaggregation over TCP serve more SLO-meeting load than tuned colocation?

Our contributions are:

- a joint evaluation of chunk size and disaggregation under per-request stall limits, with a map of where each wins as the SLO changes (Sections 4.2–4.4);
- a characterization of the KV transfer over cloud TCP: the UCX segment size, NIXL's four connections, the cloud's per-flow cap, and a transfer-time model that estimates disaggregation's TTFT cost (Sections 4.5–4.6);
- three pitfalls that change conclusions: percentile metrics hide per-request stalls, a multimodal budget check keeps Gemma 4's chunk at 2,496 tokens or more unless every modality is disabled, and the reference proxy's garbage collection produces stalls that look like disaggregation's (Sections 4.1, 4.2, 4.7);
- the harness and per-request data of all runs, including configurations that did not help.

2 Background and Related Work

Disaggregated serving. DistServe splits prefill and decode, provisions each phase for its own latency target, and measures per-GPU goodput at an SLO attainment target [2]; we use the same metric. With 25 Gbps between nodes, DistServe keeps the KV transfer on intra-node NVLink. Splitwise runs the phases on different machine types over InfiniBand and overlaps the transfer with prefill layer by layer [3]. Mooncake builds a KV-cache-centric architecture over RDMA [4]. TetriInfer [10], DéjàVu [11], P/D-Serve [12], and KVDirect, which pulls the KV cache from the decode side as vLLM's connector does [13], belong to the same family and assume a fast interconnect.

Colocation and when to split. Sarathi-Serve introduced chunked prefill and made the chunk size the knob that trades prefill throughput against decode stalls [1]. semi-PD disaggregates compute inside one GPU while sharing KV storage [14]. TaiChi unifies aggregation and disaggregation and reports that aggregation is best for tight TTFT and relaxed per-token latency, and disaggregation for strict per-token latency and relaxed TTFT [15]. Our results reproduce that boundary on TCP-connected single-GPU nodes and add the chunk size as the variable that moves it. EcoServe targets clusters with commodity interconnects, where full disaggregation depends on fast links,

and proposes partial disaggregation [16]; ThunderServe and HexGen-2 schedule phase-split serving across heterogeneous cloud GPUs and networks [17, 18].

KV transfer layer. vLLM [19] moves the KV cache with NIXL [20], which wraps transports including UCX [21]; without RDMA, UCX uses TCP. CacheGen compresses the KV cache to reduce network traffic [22]. Prefill-as-a-Service sends the small KV caches of hybrid-attention models between datacenters over commodity Ethernet [23], and PDD disaggregates across clusters linked by wide-area TCP [24]. The llm-d project measured GPU-to-GPU transfers over 100 Gbps TCP at 0.55 GB/s with UCX and 4.7–4.9 GB/s with UCCL [25]. Mitra et al. find disaggregation most effective for prefill-heavy traffic and larger models [26].

Metrics. Etalon argues that TTFT, time between tokens, and their percentiles miss what users experience and proposes a fluidity index [27]; Andes defines a quality of experience over each request's token delivery timeline [28]. We use a simpler per-request criterion, the longest gap between consecutive output tokens.

Where this work sits. We do not design a system. We measure stock vLLM on rented single-GPU nodes, compare disaggregation with colocated replicas whose chunk size we tune, and explain the transfer path. We did not find a measurement of this setting, but our search was not exhaustive.

3 Methodology

3.1 Environment

We used three single-GPU instance types in one availability zone of AWS us-east-1 (Table 1). None of these sizes supports EFA or ENA Express [5], so all traffic between nodes is TCP. Within the zone, one TCP flow reached at most 4.97 Gbps by iperf, matching the documented 5 Gbps single-flow limit [29]. Inside a cluster placement group, an AWS feature that places instances close together, one flow reached 9.54 Gbps (documented: 10 Gbps), and eight parallel flows reached 49.72 Gbps, the g7e's nominal 50 Gbps. Load generation and request routing ran on a separate CPU instance (c6i.xlarge before session E7, c6i.2xlarge in E7 and E8).

Instance	GPU	GPU memory	$/hour	Used for
g7.2xlarge	RTX PRO 4500	32 GB	–	input-length comparison, first transfer tuning
g6e.2xlarge	L40S	48 GB	2.242	cross-GPU comparison
g7e.2xlarge	RTX PRO 6000	96 GB	3.363	placement group, chunk sweep, head-to-head, scaling, repeats

Table 1: GPU instances (one GPU each, no EFA). Prices are on-demand in us-east-1; costs in this paper use them. All main-text results come from g7e.

We used vLLM v0.30.0 with NIXL's pull-based connector, in which the decode node reads the KV cache from the prefill node. Unless stated otherwise, disaggregated runs used seven UCX environment variables that we tuned in early sessions (Appendix C); Section 4.5 shows that one of them, the TCP segment size, accounts for the whole gain. All servers used a GPU memory utilization of 0.90, at most 256 concurrent sequences, and a 32,768-token context.

3.2 Models and KV size

We served two Gemma 4 models [30]. 26B-A4B is a mixture-of-experts model that activates 3.8B of its 25.2B parameters per token; 31B is dense (30.7B parameters). Both interleave sliding-window and global attention. We mainly used NVIDIA's NVFP4 checkpoints, whose KV cache is FP8 [31], and for comparison FP8 (weight-only) and BF16 checkpoints of 26B, whose KV cache is BF16. Because sliding-window layers hold only the most recent tokens, the transferred KV volume is affine rather than proportional in the input length L. For 26B NVFP4, vLLM reported 140, 260, and 412.5 MiB per request at 4K, 16K, and 32K inputs, which fits

$$V(L) \approx 100 \text{ MiB} + 10 \text{ KiB} \times L. \tag{1}$$

The FP8 and BF16 checkpoints transfer twice as much (280 MiB at 4K). 31B NVFP4 transfers 560 MiB at 4K. vLLM reports these volumes as "MB"; they are mebibytes, and we convert them to Gbps as MiB $\times 8 \times 1.048576$/s.

3.3 Configurations

We compared two approaches at equal GPU count. In the *replica* configuration, each node runs a colocated server that does both prefill and decode, and a round-robin proxy spreads requests over the nodes. Replicas use chunked prefill; the *chunk* is vLLM's maximum number of tokens per scheduler step. In the *disaggregated* configuration there are prefill-only (P) and decode-only (D) nodes, and we write x prefill and y decode nodes as xPyD. Prefill nodes used a 2,560-token chunk.

Two practical details shaped the comparison. First, for Gemma 4, vLLM rejects chunks smaller than 2,496 tokens, the budget of one multimodal item, even with images disabled; the check passes only when images, audio, and video are all disabled (`-limit-mm-per-prompt` with every modality set to 0). Our sessions before E8 therefore used 2,560 tokens; session E8 swept 256–2,048. With every modality disabled, a 2,560-token chunk failed at engine start, so the 2,560 control in E8 kept images disabled only. Second, the proxies differed. Replicas used a small aiohttp round-robin proxy; disaggregated runs before E8 used vLLM's test proxy for NIXL (FastAPI with httpx), and E8 added an aiohttp proxy that implements the same request flow (Section 4.7).

3.4 Workloads

Chat uses ShareGPT conversations [32] (median input 76–141 and median output 59–177 tokens, depending on the sampled subset). *Long input* uses synthetic requests with 4,096, 16,384, or 32,000 input tokens and 512 output tokens. *Trace* replays 600 requests of the conversation trace released with Splitwise (median input 996, output 217 tokens) [33]. Requests arrive as a Poisson process. A measurement point at a total rate of r requests/s sends $\max(100, \lceil 180r \rceil)$ requests, about 180 s at 0.56 requests/s or more. We report rates per GPU unless stated otherwise.

3.5 Metrics and SLO

We judge each request individually. Under the *strict SLO*, a request must have TTFT (time from sending the request to the first output token) of at most 1 s and a longest inter-token gap of at most 50 ms; the *relaxed SLO* uses 3 s and 250 ms. A request with a single output token has no gap and is judged on TTFT only. We use the longest gap because one long pause in a 500-token answer is 0.2% of its gaps and vanishes from percentile statistics, yet it is the pause a user sees.

Attainment is the fraction of requests that meet the SLO. *Supported load* is the highest measured rate at which every repeat reaches 0.9 attainment. *SLO goodput* is the output-token throughput of

SLO-meeting requests (the minimum over repeats at that rate), and *cost* is the GPU hourly price divided by SLO goodput, in dollars per million tokens.

The thresholds are design choices. TTFT targets in prior work range from 0.125 s to 30 s [2, 4]; 1 s is the classic limit for keeping a user's flow of thought [34]. For the gap, our pre-registered protocol used five times each model's decode step, Sarathi-Serve's strict rule [1]. We switched to a common absolute limit because a relative rule makes faster models stricter in absolute terms (38 ms for 26B BF16 on g7e) and makes models hard to compare; 50 ms is five times the 26B decode step on the slowest GPU we used. Section 4.4 shows how each verdict changes over a grid of TTFT and gap limits.

3.6 Procedure and data selection

All reported runs had the prefix cache disabled. Some early runs had it enabled while replaying the same seeded requests, so later points hit cache entries left by earlier ones; we excluded those 122 runs, 5 server-start failures, and 1 discarded run, 128 of 436 runs. Five runs in which the client-side proxy reached its CPU limit are marked as caution data and not used for verdicts. The remaining 303 runs are the *audited* runs. Sessions E7 and E8 warmed every server with 30 requests; earlier sessions did not, which changed attainment by at most 0.019 when we dropped each run's first five requests.

Repeated points used different seeds. Median TTFT varied by at most 5.1% across repeats. Points well above 0.9 attainment varied by at most 0.03; points near 0.9 varied by up to ± 0.09 and could flip a verdict, and we call them *borderline*; low points varied more (0.37–0.64). For decisive points we report 95% intervals from a time-block bootstrap (20 blocks per run). Appendix E maps sessions to data folders, and Appendix D lists deviations from the pre-registered protocol.

4 Results

4.1 System percentiles hide per-request stalls

The system-wide p99 of inter-token latency pools every gap of every request, and it can look healthy while many requests still stall (Figure 1). Of the 202 audited runs with TTFT p99 below 1 s, 28 had system p99 at or below 50 ms but attainment below 0.9. For example, two g7e replicas serving 26B FP8 with 4K inputs at 0.25 requests/s per GPU had a system p99 of 8.6 ms, yet 48% of requests had at least one gap over 50 ms. All later verdicts are per request.

4.2 The chunk size sets the replicas' stall

A replica stalls when another request's prefill joins the step, and the stall lasts as long as that step. With 4K inputs and 512 outputs almost every request overlaps another request's prefill, so its longest gap is the time of a step that carries a full chunk. For 26B NVFP4 that time fell with the chunk (Figure 2a): the median longest gap was 48.4 ms at 2,560 tokens, 38.8 ms at 1,536, and 27.6 ms at 1,024, while median TTFT rose only from 132 to 161 ms. The 2,560-token median sat 1.6 ms under the 50 ms limit, so the replicas' verdicts at that chunk hinged on a margin that the model, GPU, or load could erase. A 2,048-token chunk gave 52.3 ms, longer than 2,560; we could not tell whether the even split of a 4K input into two 2,048-token pieces or the different multimodal setting of the 2,560 control causes this.

Heavier FP8 weights make each step longer: at 2,560 tokens the median longest gap was 62 ms and attainment 0.13 at 0.5 requests/s per GPU; at 1,024 tokens it was 43 ms and attainment 0.99

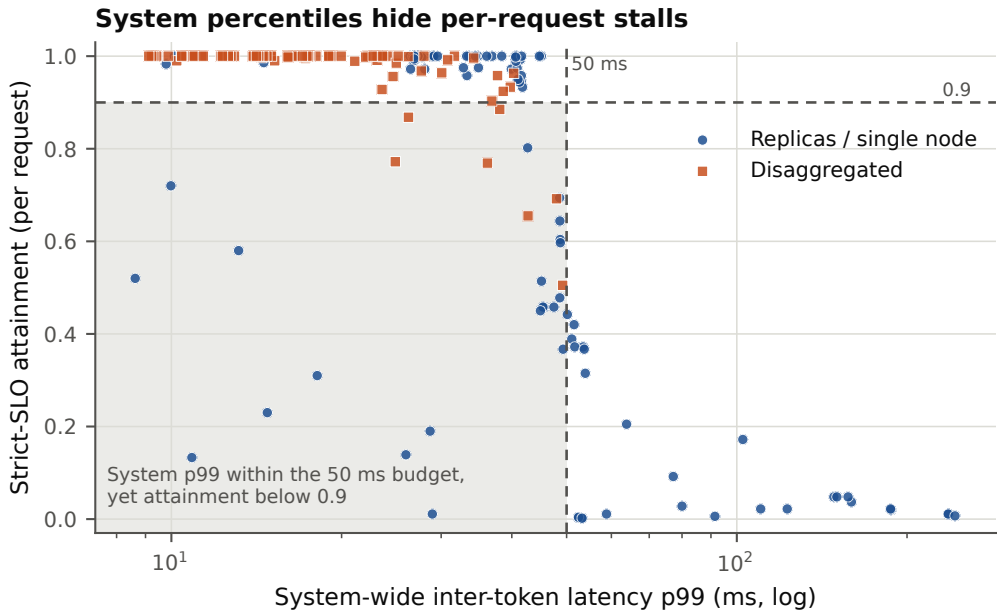


Figure 1: System-wide inter-token latency p99 (log scale) against per-request strict-SLO attainment for the 202 audited runs with TTFT p99 below 1 s. The shaded region holds runs whose system p99 is within 50 ms yet whose attainment is below 0.9: 21 replica or single-node runs and 7 disaggregated runs.

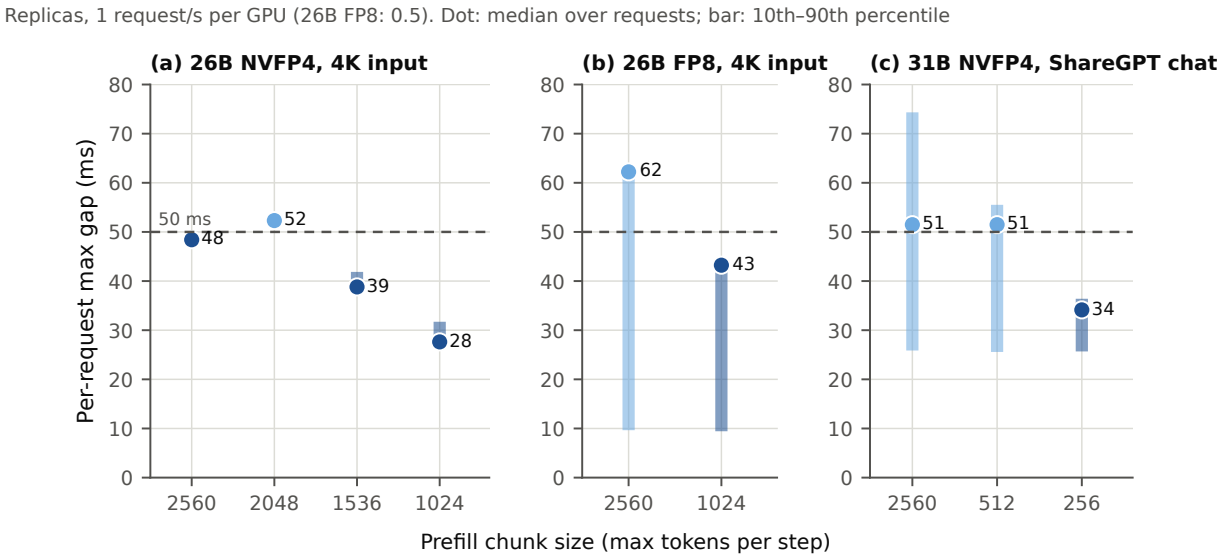


Figure 2: Per-request longest gap of colocated replicas by chunk size (two g7e nodes in a placement group). Dot: median over requests; bar: 10th–90th percentile; light dots have a median above 50 ms. (a) 26B NVFP4, 4K input, 1 request/s per GPU. (b) 26B FP8, 4K input, 0.5 requests/s per GPU. (c) 31B NVFP4, ShareGPT chat, 1 request/s per GPU. The 2,560-token points pool the repeats of sessions E5, E7, and E8.

(Figure 2b). For 31B chat, the long tail of ShareGPT prompts matters (Figure 2c). In 2,560-token runs at 1 request/s per GPU, when the longest prompt prefilled on the same replica during a request's decode had fewer than 200 tokens, 99–100% of requests kept every gap under 50 ms; with 400–800 tokens, 12% did; above 800 tokens, none did. A 512-token chunk therefore did not help (attainment 0.44, the same as 2,560), and a 256-token chunk did (1.00 in two runs, median longest gap 34 ms, TTFT unchanged at 79 ms).

4.3 Head to head at equal GPU count

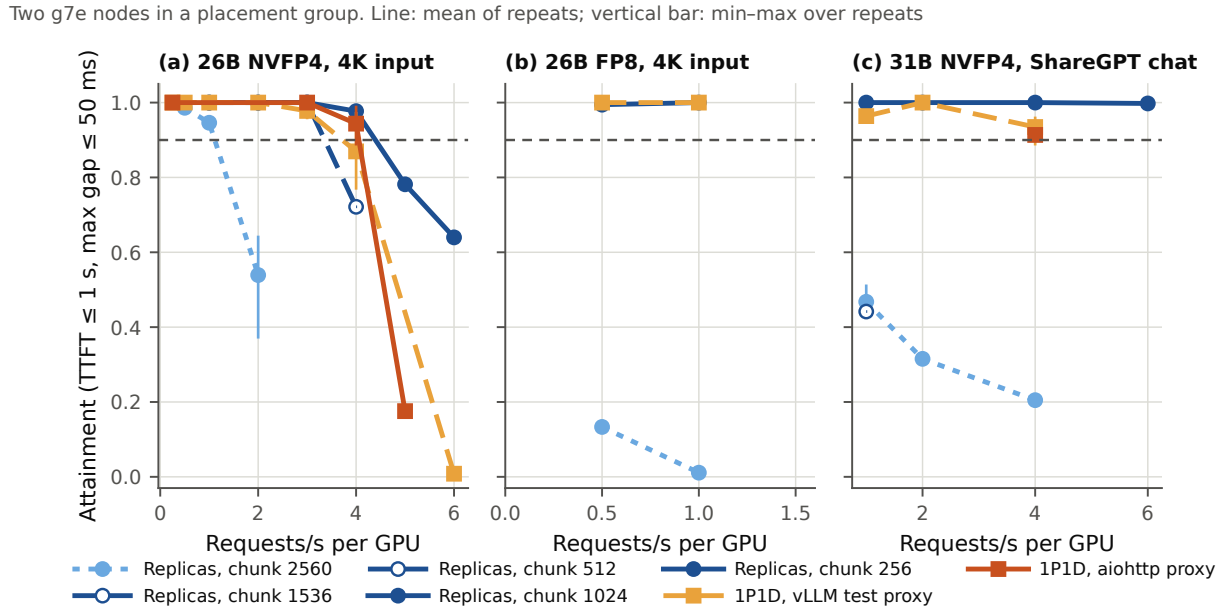


Figure 3: Strict-SLO attainment against load on two g7e nodes in a placement group. Blue: replicas at the chunk shown; orange: 1P1D with vLLM's test proxy (light, dashed) or with our aiohttp proxy (dark, solid). Lines connect the mean of repeats; vertical bars span the minimum and maximum. The dashed horizontal line is 0.9. The 2,048-token replicas (attainment ≤0.011 at every load) are omitted.

Workload	Replicas, chunk 2,560	Replicas, best chunk	1P1D
26B NVFP4, 4K	1 req/s, $2.00	4 (1,024), $0.49	3, $0.63
26B FP8, 4K	none	≥1 (1,024), ≤$1.87	≥1, ≤$1.89
31B NVFP4, chat	none	≥6 (256), ≤$0.89	4 (aiohttp proxy), ≤$1.67

Table 2: Supported load under the strict SLO (requests/s per GPU; every repeat ≥0.9) and cost per million SLO-meeting output tokens at that load (g7e, $3.363 per GPU-hour, client excluded). "≥" and "≤": the configuration still passed at the highest rate we measured. With vLLM's test proxy, 1P1D supported 2 requests/s per GPU for 31B chat ($2.58).

Measured against replicas at a 2,560-token chunk, disaggregation looked decisive (Figure 3, dotted lines). For 26B with 4K inputs, replicas passed 1 request/s per GPU and failed at 2 (0.37–0.64 in five runs), while 1P1D passed 3; the cost per SLO-meeting token was $2.00 against $0.63 (Table 2). For 26B FP8 and for 31B chat, only 1P1D met the strict SLO at any measured load.

Measured against replicas at the best chunk we tried, the result is a tie or a reversal. For 26B with 4K inputs, 1,024-token replicas passed up to 4 requests/s per GPU (0.977, interval 0.950–0.997) and failed at 5 (0.78). 1P1D passed 3 and was borderline at 4: 0.885, 0.954, and 0.992 in three runs with our aiohttp proxy, and 0.77–0.98 in four runs with the test proxy; at 5 it collapsed to 0.18 as prefills queued. Per GPU the two approaches support about the same load; the replicas deliver the first token in about 200 ms against about 500 ms for 1P1D at 4 requests/s per GPU; and at their supported loads the replicas cost $0.49 against $0.63 per million tokens. For 26B FP8, both 1,024-token replicas and 1P1D passed the highest load we measured, 1 request/s per GPU. For 31B chat, 256-token replicas passed 6 requests/s per GPU (0.998), while 1P1D passed 4 only narrowly (0.903 and 0.924 with our aiohttp proxy; 0.885–0.963 in four runs with the test proxy).

4.4 Where each approach wins

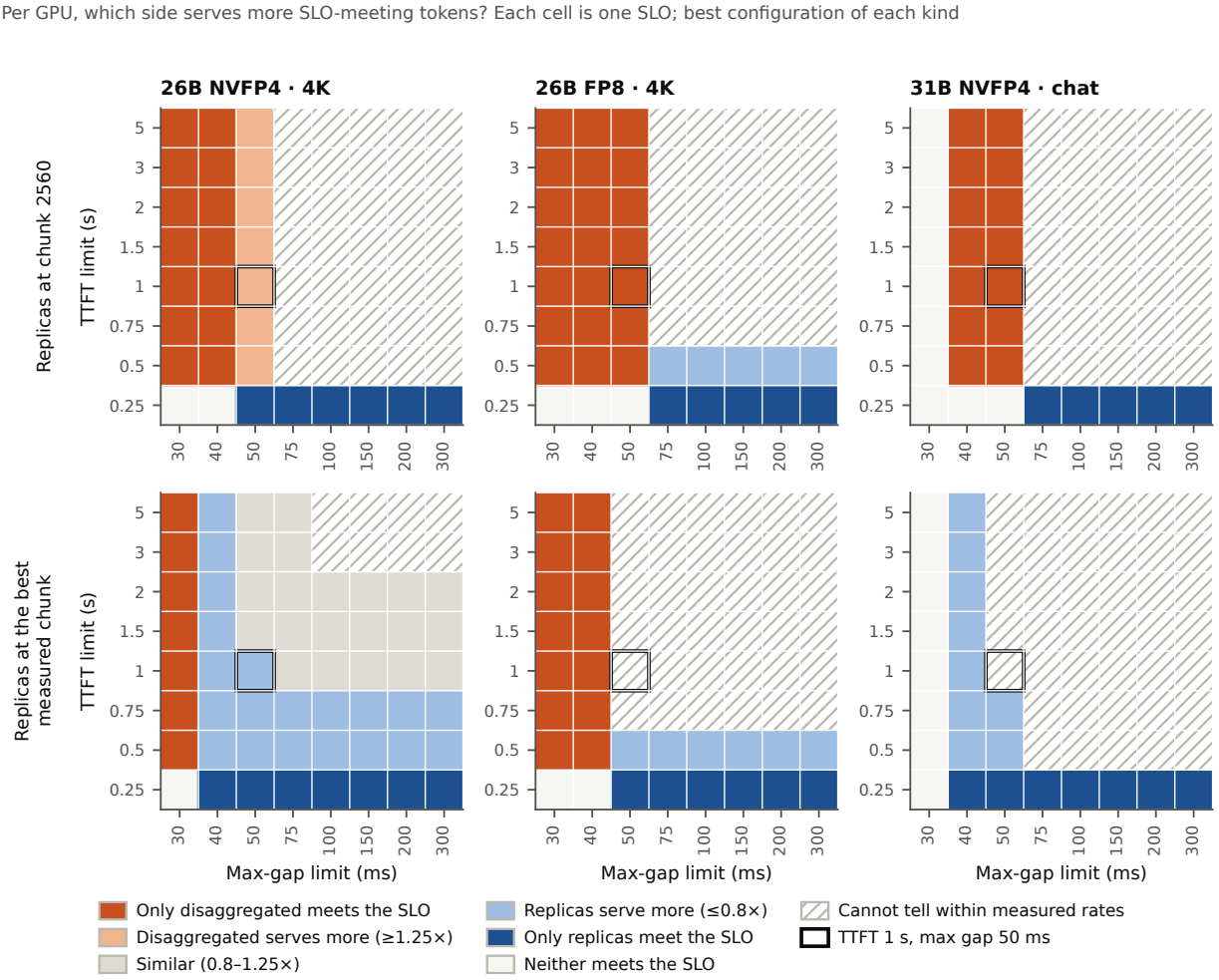


Figure 4: Which approach serves more SLO-meeting tokens per GPU as the SLO changes. Each cell is one SLO (vertical: TTFT limit; horizontal: longest-gap limit), judged at the highest rate where every repeat reaches 0.9 and with the best configuration of each kind. Top row: replicas restricted to a 2,560-token chunk. Bottom row: replicas at any chunk we measured (1,024–2,560 for 26B, 256–2,560 for 31B). Hatched: both sides still passed at their highest measured rate. The outlined cell is the strict SLO.

Because every per-request timing is stored, we re-judged all runs over a grid of TTFT limits (0.25–5 s) and longest-gap limits (30–300 ms) and compared the best configuration of each kind (Figure 4). Against 2,560-token replicas (top row), disaggregation wins at gap limits of 40–50 ms for all three workloads and at 30 ms for 26B; at 75 ms and above most cells cannot be decided within our measured rates. Against the best measured chunk (bottom row), the map changes:

- Disaggregation is the only option when the gap limit is tighter than the replicas' shortest measured stall: 30 ms for 26B NVFP4 (1P1D passes 2 requests/s per GPU; 1,024-token replicas pass nothing) and 30–40 ms for 26B FP8.
- Replicas serve more at 40 ms for 26B NVFP4 and 31B chat, and at 50 ms for 26B NVFP4; the two are similar at 75 ms and above for 26B. For 31B chat at 50 ms, replicas passed 6 requests/s per GPU against 1P1D's borderline 4, but the map marks the cell undecided because 1P1D was measured only at 4.
- Replicas are the only option when TTFT must stay under 0.25 s, because disaggregation's TTFT, which includes an 80–300 ms transfer, exceeds that limit.

Two caveats bound these statements. We did not measure chunks below 1,024 tokens for 26B, which would shorten the replicas' stall further at some cost in prefill throughput and TTFT; and several cells cannot be decided because both sides passed our highest measured rate.

4.5 The KV transfer over TCP

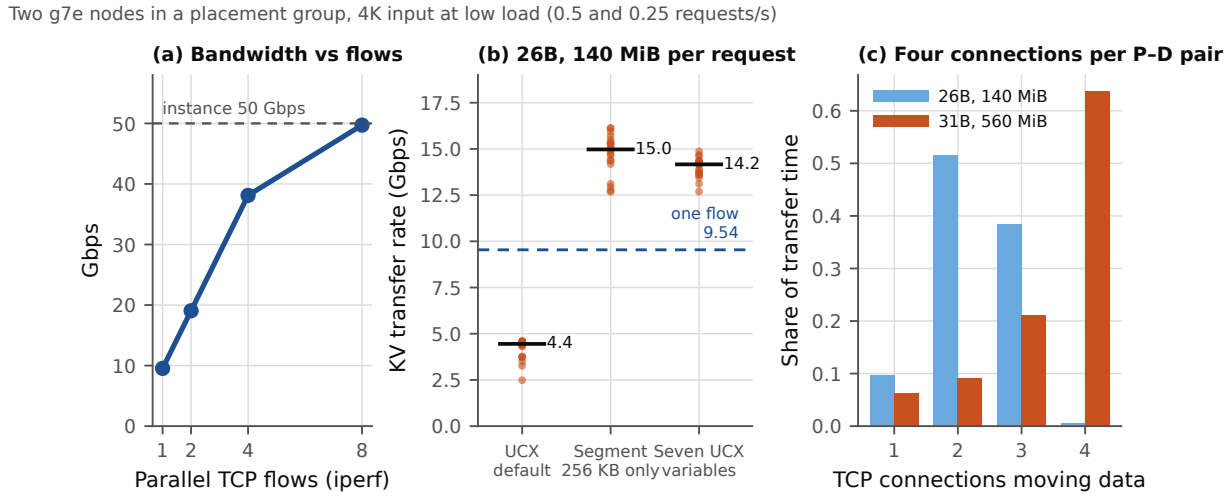


Figure 5: (a) Bandwidth between two g7e nodes in a placement group against the number of parallel TCP flows (iperf). (b) KV transfer rate of 26B NVFP4 (140 MiB per 4K request, 0.25 requests/s per GPU) under three UCX settings; dots are 10 s windows reported by vLLM, bars are medians, and the dashed line is one flow's limit. (c) Share of transfer time during which 1–4 TCP connections between the prefill and decode node moved data, from socket samples every 0.1 s (26B at 0.25 and 31B at 0.125 requests/s per GPU).

Two things set the transfer rate: a software default and the cloud's per-flow cap. With UCX defaults, the 8 KB TCP segment limited the transfer to a median of 4.45 Gbps, less than half of one flow's limit (Figure 5b). Raising only the send and receive segment sizes to 256 KB lifted it to 14.97 Gbps, a 3.4× gain; the full set of seven UCX variables gave 14.16 Gbps, no better. In our first tuning session, on g7 nodes without a placement group, the seven-variable change cut median

TTFT at 0.125 requests/s per GPU from 626 to 466 ms and raised attainment at 0.5 requests/s per GPU from 0.82 to 0.99.

The tuned rate exceeds the 9.54 Gbps limit of one flow by 1.5–1.6× for 26B and 1.8× for 31B (560 MiB at a median 17.3 Gbps) because the transfer does not use one flow. Socket samples showed four TCP connections between the prefill and decode node, each carrying 22–27% of the bytes, plus a fifth carrying 1–3% (Figure 5c). This matches the four worker threads of vLLM's NIXL connector in v0.30.0, each with its own UCX endpoint. A 26B transfer mostly kept two or three connections busy; the larger 31B transfer kept all four busy 64% of the time, which is why its rate is higher. Because the instance's aggregate bandwidth grows almost linearly with flows (Figure 5a), more connections, not other UCX settings, are the remaining lever; we did not test that.

Other settings did not help. Staging the KV cache through host memory slowed TTFT 4.7× at 0.125 requests/s per GPU on g7 and had no stable load at 0.5. Push transfer, in which the prefill node writes the KV cache, cut 31B's transfer time by 27–36% and median TTFT by 53–62 ms but changed attainment by at most 0.04; its logged volume was 468–533 MB per request against 560 for pull, which we could not explain. Within one node over PCIe, push has been reported about 100× faster than pull [35]; we saw no such gap across nodes. A proposed vLLM patch that overlaps the transfer with prefill layer by layer[1] supports only models with a single KV-cache group; with hybrid-attention Gemma 4 it stalled on the first request. Network credits were not a constraint: g7e.2xlarge has a fixed 50 Gbps, and on g6e.2xlarge (5 Gbps baseline, 20 Gbps burst) our 15-minute test load averaged about 1.2 Gbps, computed from the request rate and volume, below the baseline.

4.6 The first-token cost of disaggregation

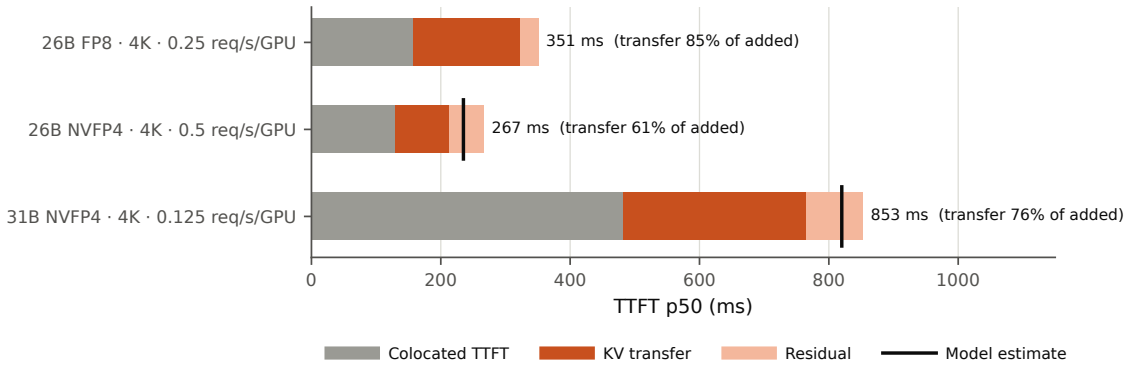


Figure 6: Median TTFT of 1P1D split into colocated TTFT at the same load, mean KV transfer time, and residual (placement group). Black ticks are estimates from Eq. (2). The first row joins measurements from two days.

Disaggregation adds TTFT, and most of it is the KV transfer (Figure 6): 61% of the added time for 26B NVFP4 at 0.5 requests/s per GPU (84 of 137 ms), 76% for 31B at 0.125 (283 of 370 ms), and 85% for 26B FP8 at 0.25. The residual, 28–87 ms, covers the decode node's first step, scheduling, and the proxy hop, and it would remain with RDMA. At low load the relation is

$$\mathrm{TTFT}_{\mathrm{PD}} \approx \mathrm{TTFT}_{\mathrm{coloc}} + \frac{V(L)}{m \times C_{\mathrm{flow}}} + \mathrm{residual}, \tag{2}$$

[1] `https://github.com/vllm-project/vllm/pull/56874`

with $V(L)$ from Eq. (1), C_{flow} the per-flow cap, and m the effective number of parallel connections (1.5–1.8 in our tuned runs). We wrote estimates from this relation into our lab notebook before the three-node session: 235 ms for 26B at 0.5 requests/s per GPU (measured 267 ms, 14% higher) and 810–830 ms for 31B at 0.125 (measured 853 ms). The repository history cannot prove that order. The relation omits queueing: when we limited the sender's aggregate bandwidth to 1 Gbps, it predicted about 1.5 s against a measured 2.19 s, and adding an M/D/1 waiting time explains the difference (about 2.2 s). Eq. (2) is therefore a low-load estimate. It also shows why long inputs fail: for 26B FP8 at 16K inputs, about 520 MiB take roughly 0.3–0.7 s on top of a prefill of about 0.8 s. We measured 1P1D's TTFT 465–506 ms above the replicas' there, beyond a 1 s budget.

4.7 A pitfall: proxy stalls that look like disaggregation's

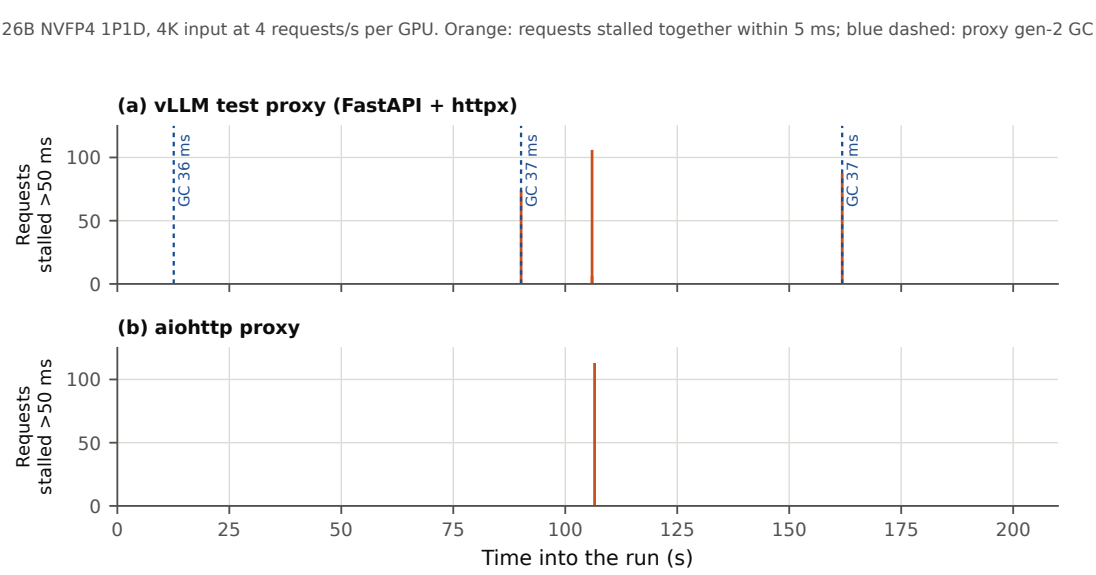


Figure 7: Global stalls in two 1P1D runs with the same seed (26B, 4K input, 4 requests/s per GPU). Orange: number of requests whose output stream paused for more than 50 ms within the same 5 ms. Blue dashed: second-generation garbage collections in the proxy. (a) vLLM's test proxy. (b) Our aiohttp proxy.

Near their limits, disaggregated runs failed in a way replicas did not: one to five times per run, 40–110 in-flight requests paused together for 55–80 ms and then received a burst of tokens. We instrumented both proxies to log upstream stream gaps, event-loop lag, and garbage collection, and we snapshotted the decode server's inter-token latency histogram before and after each point. With vLLM's test proxy, four of five such events in three runs coincided within 100 ms with a second-generation garbage collection of 25–40 ms that blocked the proxy's event loop (Figure 7a). With our aiohttp proxy, second-generation collections took 9 ms and the loop never lagged. One event remained in three runs; the proxy received it from upstream, and the decode server logged it as 113 gaps over 50 ms. The same event appears at the same time in the test-proxy run with the same seed, so it is a server-side pause tied to the request sequence.

Removing the global events raises 1P1D's attainment at 4 requests/s per GPU from 0.77–0.99 to 0.94–0.99. The test proxy therefore made our earlier disaggregated limits conservative, and it explains the collapse of our three-node 2P1D configuration at 3 requests/s per GPU (0.77, or 0.996 without the events). For 31B chat, our aiohttp proxy still showed two and four global events per run, which we did not trace further.

4.8 Other observations

These results come from sessions before E8 and used 2,560-token replicas; we report them with that caveat.

- **Three nodes.** Neither 1P2D nor 2P1D exceeded 1P1D per GPU for 26B with 4K inputs (1P2D: 0.906 at 3 requests/s per GPU), and three 2,560-token replicas passed 1 request/s per GPU. Their failures near the limit were dominated by the proxy stalls above, so we draw no rule for the prefill-to-decode ratio. For 31B with 4K inputs no configuration held a 1 s TTFT reliably: a 4K prefill takes about 480 ms and the transfer about 280 ms.
- **Input length** (g7, before transfer tuning). At 0.25 requests/s per GPU with 4K inputs, 1P1D reached 0.96 and replicas 0.23. At 16K, prefill alone pushed the replicas' TTFT over 1 s and the transfer pushed 1P1D's over, so neither met the strict SLO; under the relaxed SLO 1P1D reached 0.47 and replicas 0.17. At 32K all failed.
- **Weight precision** (g7e, same day). At 0.5 requests/s per GPU with 4K inputs, 2,560-token replicas reached 0.97 (NVFP4), 0.13 (FP8), and 0.03 (BF16), and 1P1D 1.00 in all three. Section 4.2 shows that a 1,024-token chunk rescues FP8.
- **Real trace.** Replaying the Azure conversation trace, all 26B configurations passed (4 and 5.9 requests/s in total); 1P1D's TTFT was about 120 ms higher. For 31B at 3.7 requests/s in total, both failed: 2,560-token replicas on gaps (0.05) and 1P1D on the TTFT tail (0.28).
- **Correctness.** With greedy decoding and Gemma's chat template, disaggregated 31B gave the same answers as a single node; long outputs diverged only in late, near-tied tokens (30–160 characters in), and push matched pull byte for byte on 8 of 8 prompts after a restart. 26B was not deterministic even on a single node, so we used a gate (short outputs byte-identical; long outputs sharing at least 40% of words of five or more letters, calibrated at $\geq$0.58 for true pairs and $\leq$0.17 for swapped prompts); all disaggregated outputs passed.

5 Discussion

5.1 Answers to the research questions

RQ1. The chunk size. A replica's longest gap is the time of a step that carries a full chunk of another request's prefill. Shrinking the chunk shortened it almost in proportion (26B: 48 to 28 ms from 2,560 to 1,024 tokens) at a TTFT cost of about 30 ms. For a dense model with chat traffic the chunk had to be small (256 tokens for 31B).

RQ2. The KV transfer, about 61–85% of the added TTFT at low load. Its time is the affine KV volume divided by the effective bandwidth, which the UCX segment size limits first and the per-flow cap times NIXL's connection count limits next (Eq. (2)). Queueing adds to it under load.

RQ3. When the stall limit is tighter than the shortest stall chunking achieves, or when the chunk cannot be reduced. Against replicas at the best chunk we measured, disaggregation won only at 30 ms (26B NVFP4) and 30–40 ms (26B FP8); at 40–75 ms tuned replicas served as much or more and always had lower TTFT. Against replicas stuck at a 2,560-token chunk, as a default Gemma 4 deployment is, disaggregation over TCP served about 3$\times$ more at 50 ms.

5.2 A procedure before disaggregating over TCP

1. **Measure the replicas' longest gap per request, not system percentiles.** It equals the time of a step carrying one full chunk.
2. **Shrink the chunk until that step fits the gap limit.** Check that the stack allows it (for Gemma 4 in vLLM v0.30.0, disable every modality) and that TTFT and throughput remain acceptable.
3. **Disaggregate if no acceptable chunk fits**, and check that the transfer fits the TTFT budget with Eq. (2): KV volume from Eq. (1), per-flow cap measured with one iperf flow, m between 1.5 and 2.
4. **Fix the transfer path**: raise UCX's TCP segment size (send and receive together), keep nodes in a placement group, and use a proxy that does not pause for garbage collection.

5.3 RDMA, EFA, and scope

EFA is available on the 8xlarge sizes of the same instance families, without GPUDirect RDMA on g6e and g7e [6], at roughly 1.6–2× the hourly price according to a third-party listing [7]. EFA would shrink the transfer term of Eq. (2) but not the residual, and it would not change the replicas' chunk trade-off. Where the transfer does not bind (26B, 4K), EFA is unlikely to repay the price premium; where it does (31B with 4K inputs, 16K inputs), it might. We did not measure EFA. The per-flow cap is an AWS policy, and m depends on NIXL's thread count; on other clouds, measure one flow and several flows with iperf first.

6 Limitations

- One model family (Gemma 4) with hybrid attention; models with only global attention have larger KV caches.
- One vLLM version (v0.30.0), one AWS availability zone, at most three nodes, and no RDMA or EFA baseline.
- Chunk sizes below 1,024 tokens for 26B, and 1P1D for 31B chat above 4 requests/s per GPU, were not measured; several verdicts are bounded by our highest measured rate.
- Coarse rate grids: supported loads are resolved to 1 request/s per GPU, and some decisive points have one run.
- No prompt reuse; prefix-cache hits would shorten prefills and transfers.
- Latency was measured at the client and includes the proxy; the 2,560-token replica control in E8 had a different multimodal setting than the other chunks.
- Some comparisons join sessions from different days, and sessions before E7 used a smaller client instance.

7 Conclusion

Over TCP between rented single-GPU nodes, disaggregation is neither impossible nor automatically better. Its first-token cost is mostly the KV transfer, which a single UCX setting and NIXL's parallel connections make far cheaper than the defaults suggest, and which can be estimated before

deployment. Its benefit, protecting decodes from other requests' prefills, is exactly what chunked prefill provides by other means. Measured against replicas with a well-chosen chunk, disaggregation won only when the stall limit was tighter than chunking reached. The two should be tuned together: choose the chunk first, and disaggregate when no acceptable chunk meets the stall limit.

Use of AI tools

An AI coding assistant (Claude) was used to write measurement and analysis scripts, operate the cloud infrastructure, run the mock peer review that prompted session E8, draft figures, and draft and translate the text. All experiments ran on real hardware, and the author checked the reported numbers against the raw data.

Acknowledgments

This work was carried out at the author's own expense, about $250 of cloud usage.

References

[1] Amey Agrawal et al. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024.

[2] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210, 2024.

[3] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024. doi: 10.1109/ISCA59077.2024.00019.

[4] Ruoyu Qin, Zheming Li, Weiran He, et al. Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, 2025.

[5] Amazon Web Services. Specifications for Amazon EC2 accelerated computing instances. `https://docs.aws.amazon.com/ec2/latest/instancetypes/ac.html`, 2026. Accessed 2026-09-30.

[6] Amazon Web Services. Elastic fabric adapter for AI/ML and HPC workloads on Amazon EC2 (Amazon EC2 user guide). `https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/efa.html`, 2026. Accessed 2026-09-30.

[7] Vantage. EC2 instance price listing: g7e.8xlarge and g6e.8xlarge (us-east-1, on demand). `https://instances.vantage.sh/`, 2026. Third-party listing, accessed 2026-09-30; not checked against the AWS price list.

[8] llm-d project. Disaggregated serving (documentation, v0.9). `https://llm-d.ai/docs/architecture/advanced/disaggregation`, 2026.

[9] NVIDIA. Disaggregated inference communication guide (Dynamo v1.3.0 documentation). `https://docs.nvidia.com/dynamo/v1.3.0/kubernetes-deployment/operate/disagg-communication`, 2026.

[10] Cunchen Hu, Heyang Huang, Liangliang Xu, et al. Inference without interference: Disaggregate LLM inference for mixed downstream workloads. arXiv preprint arXiv:2401.11181, 2024.

[11] Foteini Strati, Sara McAllister, Amar Phanishayee, Jakub Tarnawski, and Ana Klimovic. DéjàVu: KV-cache streaming for fast, fault-tolerant generative LLM serving. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, 2024.

[12] Yibo Jin, Tao Wang, Huimin Lin, et al. P/D-Serve: Serving disaggregated large language model at scale. arXiv preprint arXiv:2408.08147, 2024.

[13] Shiyang Chen, Rain Jiang, Dezhi Yu, Jinlai Xu, Mengyuan Chao, Fanlong Meng, Chenyu Jiang, Wei Xu, and Hang Liu. KVDirect: Distributed disaggregated LLM inference. arXiv preprint arXiv:2501.14743, 2025.

[14] Ke Hong et al. semi-PD: Towards efficient LLM serving via phase-wise disaggregated computation and unified storage. arXiv preprint arXiv:2504.19867, 2025.

[15] Chao Wang, Pengfei Zuo, Zhangyu Chen, et al. Prefill-decode aggregation or disaggregation? Unifying both for goodput-optimized LLM serving. arXiv preprint arXiv:2508.01989, 2025.

[16] Jiangsu Du, Hongbin Zhang, Taosheng Wei, et al. Efficient LLM serving on commodity GPU clusters with data-reduced cross-instance orchestration. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*, 2026. Preprint: EcoServe, arXiv:2504.18154.

[17] Youhe Jiang, Fangcheng Fu, Xiaozhe Yao, et al. ThunderServe: High-performance and cost-efficient LLM serving in cloud environments. In *Proceedings of Machine Learning and Systems (MLSys)*, 2025. arXiv:2502.09334.

[18] Youhe Jiang, Ran Yan, and Binhang Yuan. HexGen-2: Disaggregated generative inference of LLMs in heterogeneous environment. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2502.07903.

[19] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023. doi: 10.1145/3600006.3613165.

[20] NVIDIA. NIXL: NVIDIA inference xfer library. `https://github.com/ai-dynamo/nixl`, 2026.

[21] Pavel Shamis, Manjunath Gorentla Venkata, M. Graham Lopez, et al. UCX: An open source framework for HPC network APIs and beyond. In *IEEE 23rd Annual Symposium on High-Performance Interconnects (HOTI)*, pages 40–43, 2015. doi: 10.1109/HOTI.2015.13.

[22] Yuhan Liu, Hanchen Li, Yihua Cheng, et al. CacheGen: KV cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024. doi: 10.1145/3651890.3672274.

[23] Ruoyu Qin, Weiran He, Yaoyu Wang, Zheming Li, et al. Prefill-as-a-Service: KVCache of next-generation models could go cross-datacenter. arXiv preprint arXiv:2604.15039, 2026.

[24] Yida Wang, Xiuhong Li, Jianping Ma, Gan Sun, Yunshen Xu, et al. PDD: Unleashing economical and flexible heterogeneous LLM inference via cross-datacenter prefill-decode disaggregation. arXiv preprint arXiv:2609.13161, 2026.

[25] Pravein Govindan Kannan, Liran Schour, Aleksander Slominski, Raj Joshi, Nicolò Lucchesi, Carlos Costa, Moein Khazraee, and Omri Kahalon. Networking for distributed inference in llm-d. llm-d blog, `https://llm-d.ai/blog/networking-for-distributed-inference-llm-d`, 2026. Published June 23.

[26] Tiyasa Mitra et al. Beyond the Buzz: A pragmatic take on inference disaggregation. arXiv preprint arXiv:2506.05508, 2025.

[27] Amey Agrawal, Anmol Agarwal, Nitin Kedia, et al. Etalon: Holistic performance evaluation framework for LLM inference systems. arXiv preprint arXiv:2407.07000, 2024.

[28] Jiachen Liu, Jae-Won Chung, Zhiyu Wu, et al. Andes: Defining and enhancing quality-of-experience in LLM-based text streaming services. arXiv preprint arXiv:2404.16283, 2024.

[29] Amazon Web Services. Amazon EC2 instance network bandwidth (Amazon EC2 user guide). `https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-network-bandwidth.html`, 2026.

[30] Google. Gemma 4 model card. `https://ai.google.dev/gemma/docs/core/model_card_4`, 2026.

[31] NVIDIA. Gemma-4-26B-A4B-NVFP4 and Gemma-4-31B-IT-NVFP4 model checkpoints. `https://huggingface.co/nvidia/Gemma-4-26B-A4B-NVFP4`, `https://huggingface.co/nvidia/Gemma-4-31B-IT-NVFP4`, 2026.

[32] ShareGPT_Vicuna_unfiltered: ShareGPT_V3_unfiltered_cleaned_split.json. `https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered`, 2023.

[33] Microsoft Azure. Azure LLM inference trace 2023 (AzurePublicDataset). `https://github.com/Azure/AzurePublicDataset/blob/master/AzureLLMInferenceDataset2023.md`, 2023.

[34] Jakob Nielsen. *Usability Engineering*. Academic Press, 1993. Chapter 5; summarized in "Response Times: The 3 Important Limits", Nielsen Norman Group.

[35] vLLM project. Issue #52607: vLLM's default NIXL connector is ~100× slower than the push connector on non-RDMA hardware. `https://github.com/vllm-project/vllm/issues/52607`, 2026.

A Additional figures (sessions before E8; replicas at a 2,560-token chunk)

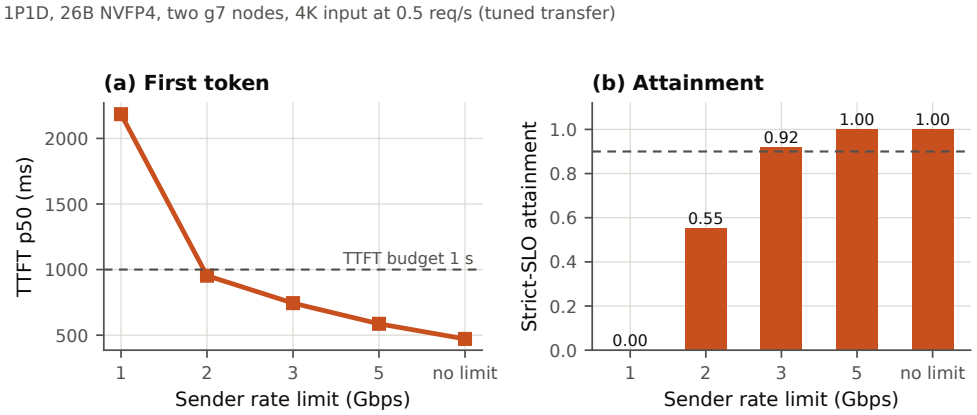


Figure A1: 1P1D on two g7 nodes, 26B NVFP4, 4K input at 0.25 requests/s per GPU, with the sender's aggregate bandwidth limited by a token-bucket filter. (a) Median TTFT. (b) Strict-SLO attainment. After transfer tuning.

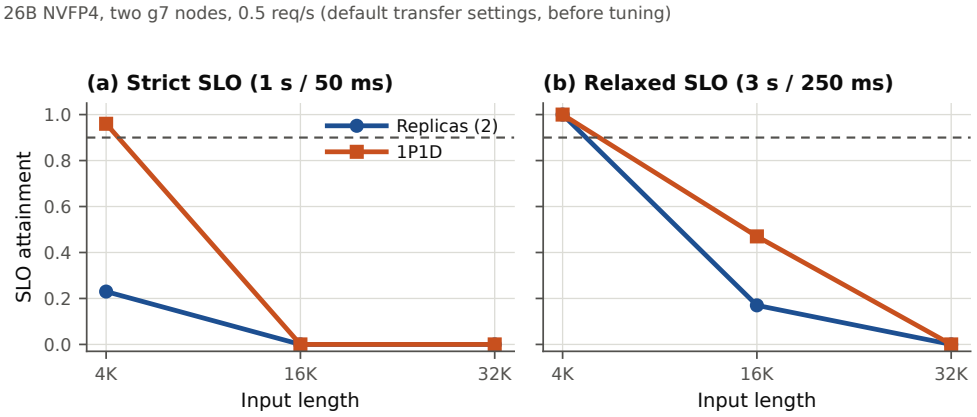


Figure A2: Attainment by input length on two g7 nodes, 26B NVFP4, 0.25 requests/s per GPU, before transfer tuning. (a) Strict SLO. (b) Relaxed SLO.

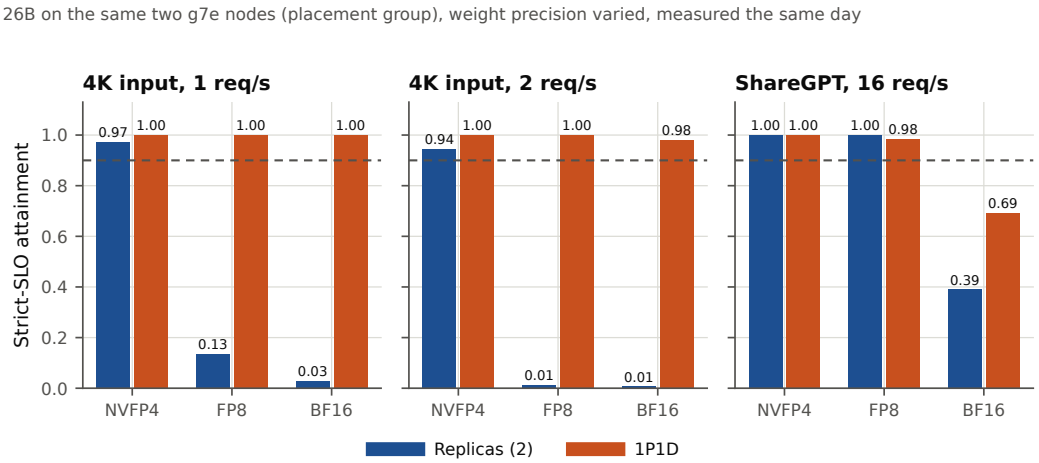


Figure A3: Weight precision varied on the same two g7e nodes in a placement group, 26B, same day, 2,560-token replicas. Rates in the panel titles are totals for two GPUs.

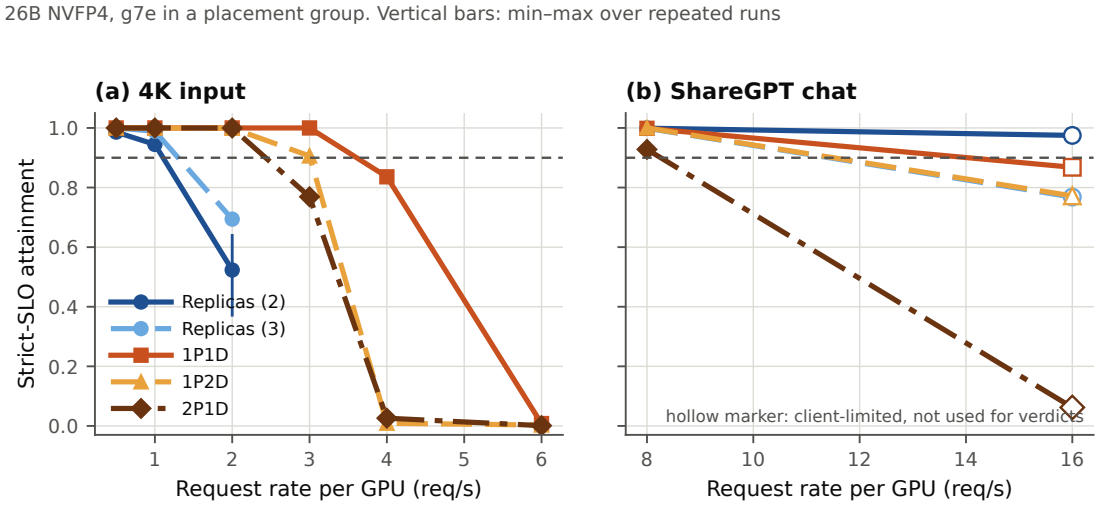


Figure A4: 26B NVFP4 on g7e in a placement group with two and three nodes (sessions E5 and E7, test proxy, 2,560-token replicas). (a) 4K input. (b) ShareGPT chat; hollow markers are client-limited.

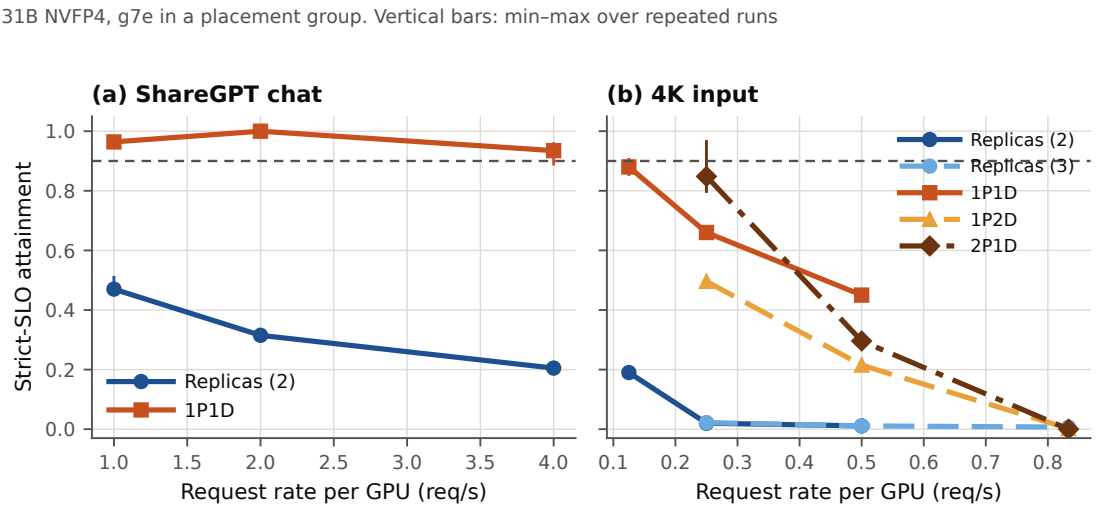


Figure A5: 31B NVFP4 on g7e in a placement group with two and three nodes (sessions E5 and E7). (a) ShareGPT chat. (b) 4K input.

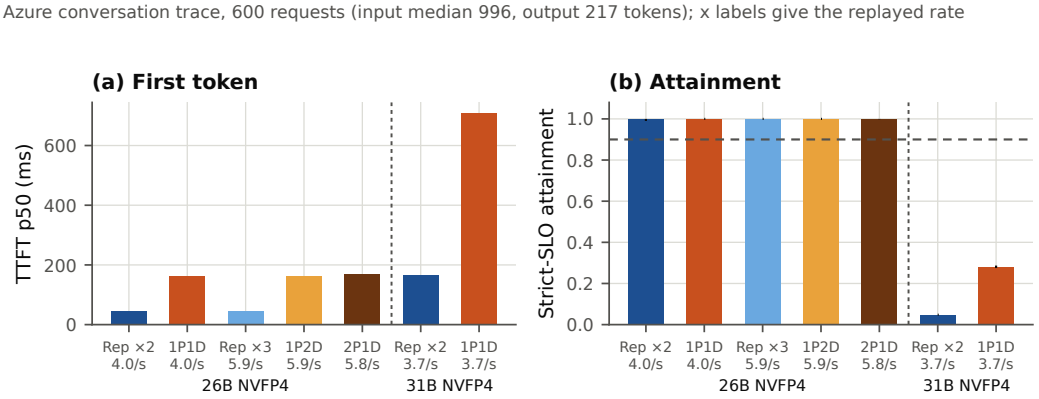


Figure A6: Azure conversation trace, 600 requests. (a) Median TTFT. (b) Strict-SLO attainment. Numbers under the axis give the replayed rate in total requests/s; left of the dotted line is 26B, right is 31B.

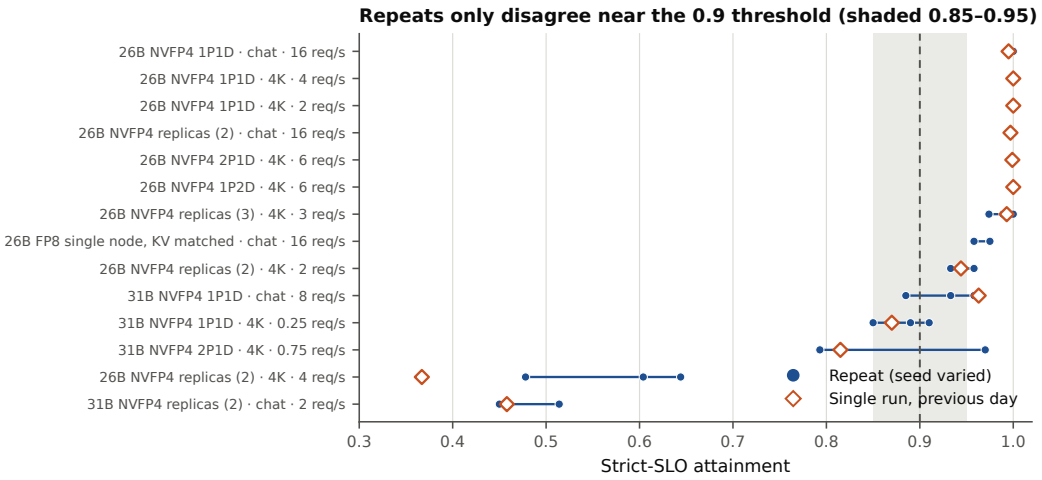


Figure A7: Attainment at points measured at least three times with different seeds (session E7) and the previous day's run (E5). The gray band is 0.85–0.95.

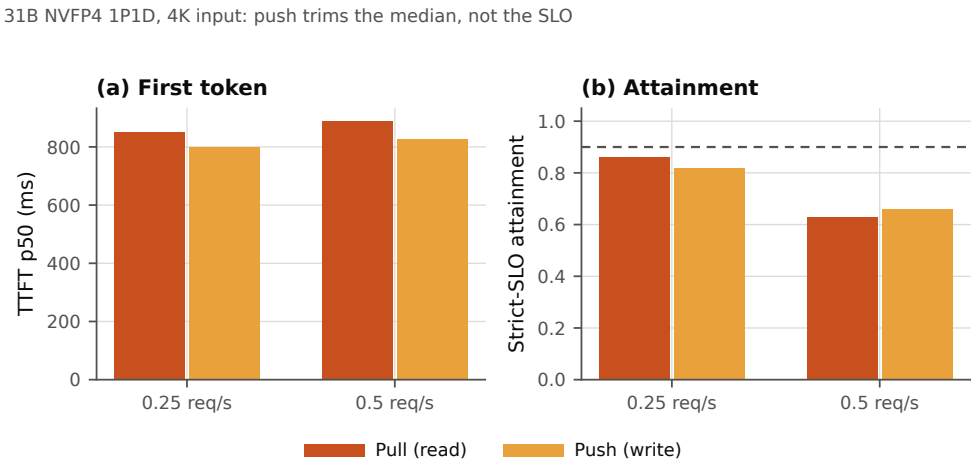


Figure A8: 31B NVFP4 1P1D, 4K input, pull versus push transfer. (a) Median TTFT. (b) Strict-SLO attainment.

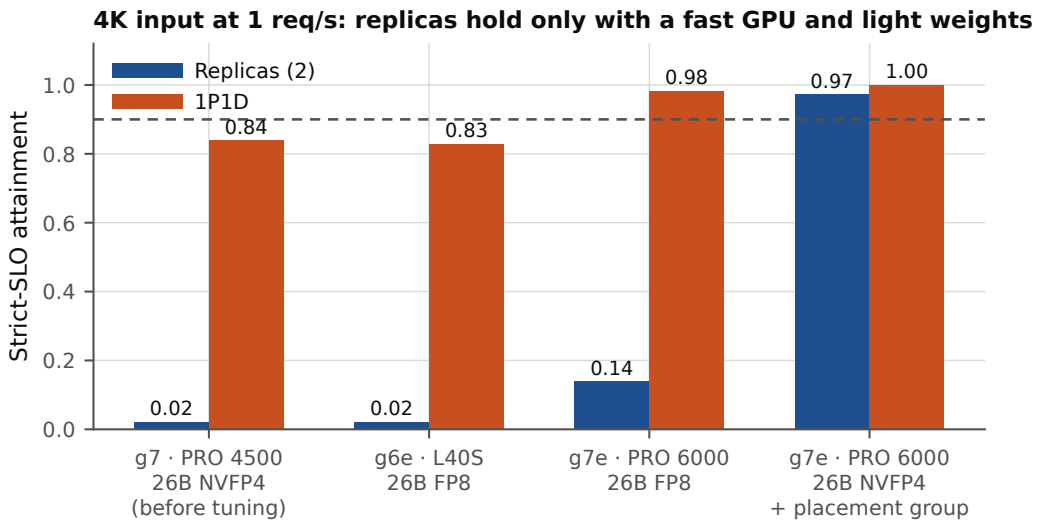


Figure A9: Strict-SLO attainment of two-node configurations at 4K input and 0.5 requests/s per GPU by GPU and precision. The g7 measurement predates transfer tuning.

B Practical pitfalls

- Gemma 4 in vLLM v0.30.0 rejects chunks below 2,496 tokens unless images, audio, and video are all disabled; with all three disabled, a 2,560-token chunk fails at engine start.
- If UCX's TCP receive segment is smaller than the send segment, the engine refuses to start; raise both.
- At high rates the proxy hit the operating system's file-handle limit (1,024 by default); a disaggregated proxy opens two connections per request and hits it sooner.
- vLLM's test proxy for NIXL pauses all streams during garbage collection (Section 4.7); our aiohttp proxy had one keep-alive race in 1,440 requests.
- Trace replay in vLLM v0.30.0 without an explicit request count builds an empty request list.
- Replaying the same seeded requests against a server with prefix caching enabled makes later points hit the cache.
- New cloud workspaces create separate security groups; the client needs an explicit ingress rule to reach new nodes.

C UCX settings

The tuned set used in sessions E4–E8:

- `UCX_TCP_TX_SEG_SIZE=256k`, `UCX_TCP_RX_SEG_SIZE=256k`
- `UCX_TCP_MAX_IOV=16`, `UCX_TCP_MAX_POLL=-1`
- `UCX_TCP_SNDBUF=8m`, `UCX_TCP_RCVBUF=8m`
- `UCX_RNDV_FRAG_SIZE=host:4M,cuda:8M`
- with `UCX_TLS=tcp,cuda_copy,cuda_ipc` and `UCX_NET_DEVICES=all` in every disaggregated run

Session E8 showed that the two segment sizes alone give the same rate (Section 4.5). We did not change kernel socket limits, so the 8 MB buffers may have been capped by `net.core.wmem_max`.

D Deviations from the pre-registered protocol

- Gap limit: five times each model's decode step became a common 50 ms (Section 3.5).
- Chunk size: 2,048 was planned; Gemma 4 forced 2,560 until session E8 swept 256–2,048.
- Proxy: one proxy for both approaches was planned; the replicas and the disaggregated runs used different proxies until E8 added an aiohttp proxy for disaggregation.
- Equivalence band: 0.9–1.1 was planned; Figure 4 uses 0.8–1.25.
- Confidence intervals: a request-level bootstrap was planned; we report time-block bootstrap intervals for decisive points only.
- Client: a fixed c6i.2xlarge was planned; sessions before E7 used c6i.xlarge. Warm-up was implemented from E7 on.
- Reference points on H100 and on a two-GPU g5 node were not measured.

E Data and code

We release the measurement harness, configurations, analysis scripts, the mock review of our first draft, and per-request raw data [GitHub URL and Zenodo DOI]. Raw data are stored per session (`results/<session>/`).

Session	Date (UTC)	Instances	Models	Main content
B	2026-09-25	2×g7	26B NVFP4	input-length grid
D	2026-09-26	2×g7	26B NVFP4	transfer tuning, bandwidth limit
E1	2026-09-26	1×g6e, 1×g7e	26B FP8, 31B FP8	cross-GPU, KV matching
E2	2026-09-26	2×g7e	26B FP8	two-node comparison, 16K
E3	2026-09-27	2×g6e	26B FP8	L40S pair, network credits
E4	2026-09-27	2×g7e, PG	26B FP8	placement group, transfer settings
E5	2026-09-27	2–3×g7e, PG	26B, 31B NVFP4	scaling, transfer estimates
E6	2026-09-27	2×g7e, PG	26B, 31B NVFP4	push transfer, correctness
E7	2026-09-28	2–3×g7e, PG	26B NVFP4/FP8/BF16, 31B NVFP4	repeats, trace, precision
E8	2026-09-29/30	2×g7e, PG	26B NVFP4/FP8, 31B NVFP4	chunk sweep, proxy instrumentation, socket counts, UCX ablation

Table E1: Measurement sessions (PG: cluster placement group). Sessions A, A′, C1, and P are prefix-cache-contaminated or setup runs and are not used.